



ITI · INTRODUCTION TO PYTHON TRACK

Python Programming for AI Applications

Day 1 — Foundations: from first lines of code to the building blocks every AI engineer needs

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications
Session	Day 1 of 6
Focus	Python history, syntax, variables, data types, control flow & functions

Course Outline

Over the next six days, we'll build a solid Python foundation and steadily point it toward AI and machine learning use cases. The roadmap below expands on the original six-day plan with the specific AI-track topics each day feeds into.

Day	Core Topics	Where it shows up in AI work
Day 1	Python history · compilers vs. interpreters · syntax · variables & data types · conditions & loops · functions	Every AI script starts here — data cleaning loops, config variables, helper functions
Day 2	Scopes · modules & packages · virtual environments (venv/conda) · pip	Isolating project dependencies (PyTorch vs. TensorFlow versions) without conflicts
Day 3	Introduction to OOP · classes, objects, inheritance	Every neural network layer in PyTorch/Keras is a class you build or extend
Day 4	File I/O · exception handling · working with JSON/CSV	Loading datasets, saving model checkpoints, parsing config files
Day 5	NumPy fundamentals · arrays & vectorized operations · intro to pandas	The backbone of numerical computing under every ML framework



Day	Core Topics	Where it shows up in AI work
Day 6	Jupyter notebooks · debugging & profiling · Git basics · intro to scikit-learn	The daily workflow of an applied ML engineer, end to end

Skills you'll be able to use by the end of the track:

- Read and write clean, idiomatic Python for data pipelines
- Structure a project using functions, modules, and classes
- Work confidently with NumPy arrays and pandas DataFrames
- Debug real programs and manage dependencies with virtual environments
- Understand enough OOP to read (and extend) PyTorch / TensorFlow model code
- Use Git for version control and Jupyter for exploratory analysis

Day 1 Outline

- Python History
- Compilers vs. Interpreters
- Python Usage
- Python Syntax
- Variables and Data Types
- Conditions and Loops
- Functions

01

Python History

Python is a widely-used, general-purpose, high-level programming language — and today it's the language of choice for most AI and machine learning work.

- Initially designed by Guido van Rossum in 1991 and developed by the Python Software Foundation.
- Python 3.10.4 is the latest stable version referenced in this course.
- Its clean syntax and huge ecosystem (NumPy, pandas, scikit-learn, PyTorch, TensorFlow) are exactly why it dominates AI development today.

From Hobby Project to the Language of AI

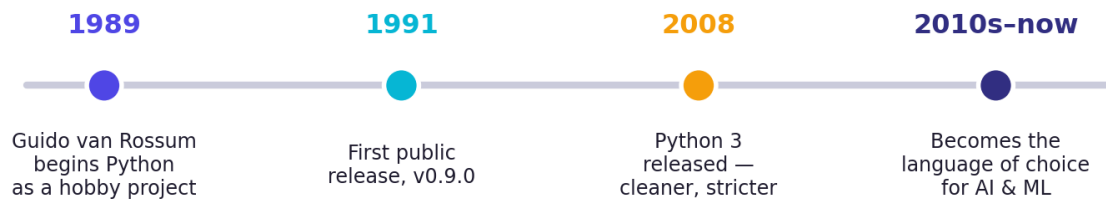


Figure 1.1 — Python's path from a 1989 hobby project to the default language of AI.

"Python has been an inspiration for many other coding languages such as Ruby, Cobra, Boo, CoffeeScript, ECMAScript, Groovy, Swift, Go, OCaml, Julia, etc."

Why Python?!

- Easy to learn — readable syntax that keeps focus on logic, not boilerplate
- Rapid development — prototype an idea (or a model) in minutes
- General purpose language — web, automation, data, and AI in one language
- Massive AI/ML ecosystem — the default language for machine learning and deep learning

✓ QUICK CHECK Python History

Q1. Who originally designed Python, and in what year did work begin?

Answer: Guido van Rossum began working on Python in 1989 (first released in 1991).

Q2. Name two libraries that make Python dominant in AI/ML today.

Answer: Any two of: NumPy, pandas, scikit-learn, PyTorch, TensorFlow.

Q3. Why does Python's readability matter specifically for AI experimentation?

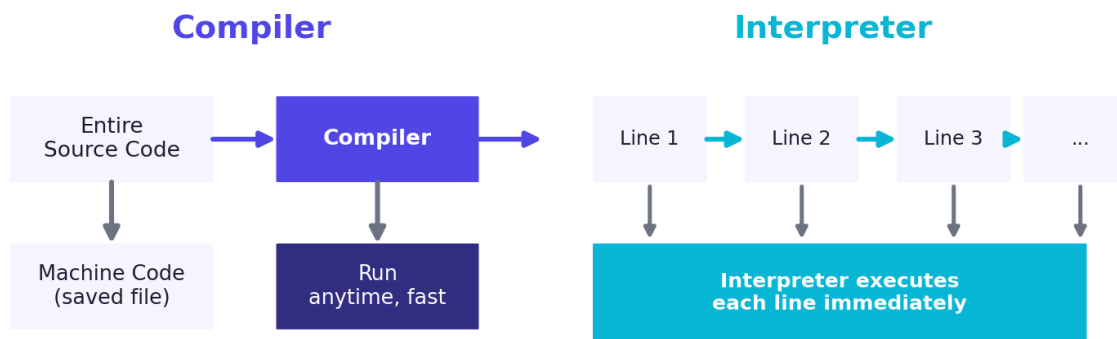
Answer: It keeps focus on the logic of an idea (or a model) rather than boilerplate, so you can prototype and iterate faster.

02

Compilers vs. Interpreters

Understanding how Python code actually runs helps explain why it's so well suited to fast, iterative AI experimentation.

	Compiler	Interpreter
Execution	Takes the entire program in one go.	Takes a single line of code at a time.
Output	Generates an intermediate machine code.	Never produces intermediate machine code.
Usage	Best suited for production environments.	Best suited for software development / experimentation.
Languages	C, C++, C#, Java, Scala	Python, PHP, Perl, Ruby



C, C++, Java, Scala — best for production

Python, PHP, Ruby — best for fast, iterative experimentation

Figure 2.1 — A compiler processes the whole program before running it; an interpreter runs line by line.

Python is interpreted — this quick feedback loop is exactly why data scientists and AI engineers can test an idea, tweak a model, and re-run it almost instantly.

AI CONNECTION Every time you change one line in a Jupyter notebook cell and re-run it without restarting the whole script, you're relying on Python's interpreted, line-by-line execution model.

```
# Try this in a Python shell (interpreter) one line at a time:
x = 10
print(x)
x = x + 5
print(x) # interpreter re-evaluates immediately, no recompilation
```

Each line executes the moment you press Enter — no separate build step.

✓ QUICK CHECK Compilers vs. Interpreters

Q1. Does a compiler or an interpreter process the entire program before running any of it?

Answer: A compiler — it takes the entire program in one go and produces machine code.

Q2. Why is Python's interpreted nature useful for AI experimentation?

Answer: It lets you test an idea, tweak a model, and re-run it almost instantly, without a separate compile step.

Q3. Name one compiled language and one interpreted language from the table.

Answer: Compiled: C, C++, C#, Java, or Scala. Interpreted: Python, PHP, Perl, or Ruby.

03

Python Usage

Python's versatility is what makes it the backbone of modern AI systems:

- Data analysis
- Machine learning & deep learning
- Web development (serving AI models via APIs)
- Automation or scripting
- Software testing and prototyping
- Everyday tasks

Where Python Shows Up

Figure 3.1 — The same language, six very different jobs.

A tiny example of each

```
# Data analysis: quick stats on a list of sensor readings
readings = [21.5, 22.1, 21.8, 23.0, 22.4]
average = sum(readings) / len(readings)
print("Average temperature:", average)

# Automation: rename every .txt file to lowercase (illustrative)
import os
for filename in os.listdir("."):
    if filename.endswith(".txt"):
        os.rename(filename, filename.lower())
```

Same language, two completely different jobs: crunching numbers and automating file tasks.

✓ QUICK CHECK Python Usage

Q1. List three areas where Python is commonly used beyond AI/ML.

Answer: Any three of: data analysis, web development, automation/scripting, software testing, everyday tasks.

Q2. How can Python be used to 'serve' an AI model to other applications?

Answer: Through web development — wrapping the model in a web API that other programs can call.

04 Python Syntax — Identifiers

A Python identifier is a name used to identify a variable, function, class, module, or other object.

Rule	Detail
Starts only with	a–z, A–Z, or _
Can contain	digits, a–z, A–Z, or _
Can't contain	punctuation characters

```
# Valid identifiers
model_name = "GPT"
_hidden_layer = 128
layer2 = "dense"

# Invalid identifiers (would raise a SyntaxError)
# 2layer = "dense"      -> can't start with a digit
# model-name = "GPT"    -> hyphens aren't allowed
```

Rule of thumb: if it could also be a variable name in math or English, it's probably valid.

Reserved Words

An identifier can't be one of Python's reserved words:

and	exec	not	assert
finally	or	break	for
pass	class	from	print
continue	global	raise	def
if	return	del	import
try	elif	in	while
else	is	with	except
lambda	yield		

Line Indentation

Python uses indentation — not braces — to define code blocks. Consistent indentation is required.

```
import time # statement 1

def count(limit): # statement 2
    result = 0 # statement 3
    for a in range(1, limit + 1): # statement 4
        for b in range(a + 1, limit + 1):
            for c in range(b + 1, limit + 1):
                if c * c > a * a + b * b:
                    break
                if c * c == (a * a + b * b):
                    result += 1
    return result
```

Each nested block (for, if) is indented one more level — Python reads the indentation as the block structure.

AI CONNECTION Inconsistent indentation is one of the most common bugs beginners hit when copy-pasting AI/ML code from tutorials — mixing tabs and spaces will raise an `IndentationError`.

Comments

```
# this is comment syntax in Python

'''
a way to declare a
string paragraph — often used as a
multi-line comment or a docstring
'''

def train_model():
    """This triple-quoted string right after a function
    definition is called a docstring — it documents
    what the function does."""
    pass
```

Docstrings (triple-quoted strings right after a `def`) are how most ML libraries document their functions.

✓ QUICK CHECK Python Syntax

Q1. Can a Python identifier start with a digit? Why or why not?

Answer: No — identifiers can only start with a letter or underscore, never a digit.

Q2. What does Python use instead of braces { } to define a code block?

Answer: Indentation (consistent whitespace).

Q3. What's the difference between a single-line comment and a docstring?

Answer: A single-line comment starts with # and is ignored by Python; a docstring is a triple-quoted string used to document a function/class and can be read by tools at runtime.

05

Variables and Data Types

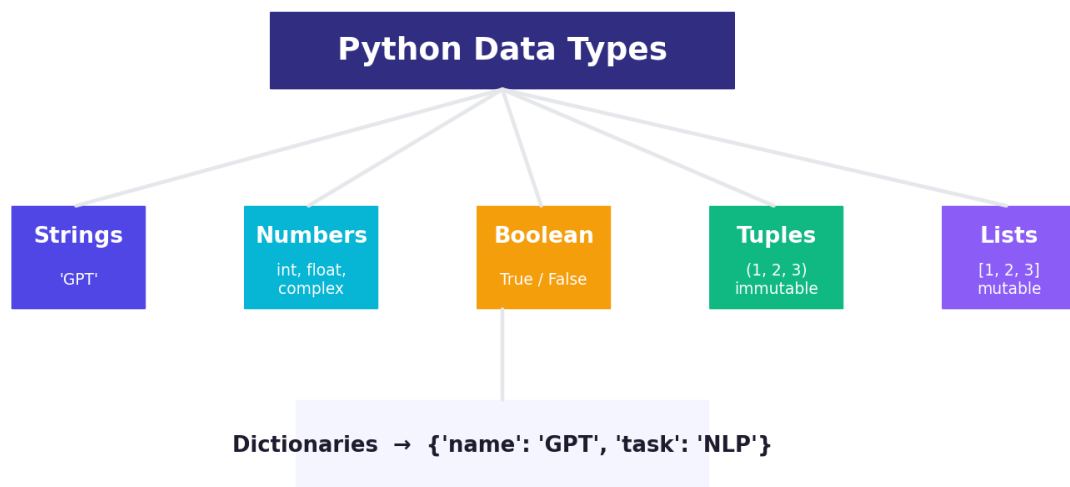
Variable Identifier = Variable Value

```
model_name = 'GPT'  
accuracy = 95  
is_trained = True  
accuracy = "ninety-five" # Python variables can be reassigned to a new type
```

Python is dynamically typed: the same name can point to an int, then a string, with no declaration needed.

Data Types

Type	Description
Strings	text data, e.g. 'GPT'
Numbers	int, float, complex
Boolean	True / False
Tuples	ordered, immutable collection
Lists	ordered, mutable collection
Dictionaries	key : value pairs



key : value pairs — the structure behind most ML feature configs

Figure 5.1 — Every Python value falls into one of these buckets.

Number Conversion

```
x = 5.16
print(int(x))      # 5      -> truncates the decimal
print(float(x))    # 5.16   -> already a float
print(str(x))      # "5.16" -> useful for building log messages

# More conversions you'll use constantly in data pipelines:
print(int("42"))   # 42
print(float("3.14")) # 3.14
print(bool(0))      # False
print(bool("non-empty")) # True
```

Type conversion ('casting') is everywhere in data loading — CSV columns often arrive as strings and need casting to numbers.

AI CONNECTION When you load a CSV with a library like pandas, numeric-looking columns can still come in as strings — `int()/float()` conversions (or pandas' own dtype casting) are what fix that before feeding data into a model.

✓ QUICK CHECK Variables and Data Types

Q1. What happens in Python if you assign a string to a variable that previously held an integer?

Answer: Nothing breaks — Python is dynamically typed, so the variable simply now refers to the string.

Q2. Which two data types are ordered but differ in mutability?

Answer: Lists (mutable) and tuples (immutable) — both are ordered collections.

Q3. Convert the string "7" to an integer and add 3 to it. What's the result?

Answer: `int("7") + 3 = 10`.

06

Operators

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
*	Multiplication	$x * y$
/	Division	x / y
%	Modulus	$x \% y$
**	Exponentiation	$x ** y$
//	Floor division	$x // y$

```
x, y = 17, 5
print(x + y) # 22
print(x % y) # 2 -> remainder, useful for FizzBuzz-style checks
print(x ** 2) # 289 -> squaring, common in loss functions (MSE)
print(x // y) # 3 -> integer division, useful for batch-count math
```

Modulus (%) and floor division (//) show up constantly when splitting datasets into batches.

Assignment Operators

Operator	Example	Same As
=	$x = 5$	$x = 5$
+=	$x += 3$	$x = x + 3$
-=	$x -= 3$	$x = x - 3$
*=	$x *= 3$	$x = x * 3$
/=	$x /= 3$	$x = x / 3$
%=	$x \% = 3$	$x = x \% 3$
//=	$x //= 3$	$x = x // 3$
**=	$x ** = 3$	$x = x ** 3$

```
total_loss = 0
for batch_loss in [0.42, 0.31, 0.28]:
    total_loss += batch_loss  # accumulating loss across batches

print(round(total_loss, 2))  # 1.01
```

`+=` is the pattern behind almost every 'running total' you'll write, including training-loss accumulation.

Comparison Operators

`a <op> b`

Operator	Meaning
<code>==</code>	returns True if a equals b
<code>>=</code>	returns True if a equals or is greater than b
<code><=</code>	returns True if a equals or is lesser than b
<code>!=</code>	returns True if a not equals b
<code>></code>	returns True if a is greater than b
<code><</code>	returns True if a is lesser than b

Logic Gates

```
# and -> AND Logic Gate
# or -> OR Logic Gate
# not -> NOT Logic Gate

print(True and False)      # False
print(True or False)       # True
print(not False)           # True
print(not (True == 2))     # True
print((False == 0) and (True == 1))  # True

# A realistic AI-flavored check:
confidence = 0.87
is_flagged = False
should_alert = (confidence > 0.8) and (not is_flagged)
print(should_alert)  # True
```

Combining comparisons with `and` / `or` / `not` is exactly how you'd write a model-confidence alert rule.

✓ QUICK CHECK Operators

Q1. What does `x // y` return compared to `x / y`?

Answer: `//` returns the floor (integer) division result, dropping any remainder; `/` returns the full decimal result.

Q2. Rewrite `total = total + 5` using an assignment operator.

Answer: `total += 5`

Q3. Evaluate: `(3 > 2)` and `(4 < 1)`

Answer: `False` — the first part is `True`, but the second part is `False`, and `'and'` requires both to be `True`.

07

Strings & Data Structures

Declaration of a string variable:

```
s = "this is a string"
s = 'this is a string too'

# Strings support handy operations you'll use in text preprocessing:
name = "gpt"
print(name.upper())      # "GPT"
print(name.capitalize()) # "Gpt"
print(len(name))         # 3
print(name + "-4")       # "gpt-4"
print(name * 3)          # "gptgptgpt"
print("pt" in name)      # True
```

String methods like `.upper()` and `.strip()` are the first tools you reach for when cleaning text data.

Data Structures — List

A collection of various data types.

```
features = [0.82, 'cat', True]
print(features[0]) # 0.82
print(features[1]) # 'cat'
print(features[2]) # True
# print(features[3]) # IndexError: List index out of range

# Lists are mutable – you can grow, shrink, and edit them in place:
scores = [0.71, 0.85, 0.63]
scores.append(0.90)
scores[0] = 0.75
print(scores) # [0.75, 0.85, 0.63, 0.90]
```

A batch of model predictions is almost always stored and manipulated as a list.

Data Structures — Tuples

Same as lists, but tuples are immutable.

```
t = (1, 'hi', True)
print(t[1]) # hi
# t[1] = 4
# TypeError: 'tuple' object does not support item assignment

# Tuples are perfect for fixed shapes that should never change by accident:
image_shape = (224, 224, 3) # height, width, channels
print(image_shape[0]) # 224
```

Image dimensions and model input shapes are classic examples of 'this should never change' — a tuple.

Data Structures — Dictionaries

A key : value, comma-separated element data structure.

```
model = {'name': 'GPT', 'task': 'NLP'}
print(model['name'])
# 'GPT'

model['name'] = 'BERT'
# {'name': 'BERT', 'task': 'NLP'}

# Dictionaries are how you'll store model configs and hyperparameters:
config = {
    'learning_rate': 0.001,
    'batch_size': 32,
    'epochs': 10,
}
for key, value in config.items():
    print(key, "->", value)
```

Hyperparameter configs in almost every ML framework are just Python dictionaries under the hood.

List vs. Tuple vs. Dictionary — at a glance

	List	Tuple	Dictionary
Syntax	[1, 2, 3]	(1, 2, 3)	{'k': 'v'}
Mutable?	Yes	No	Yes (values)
Ordered?	Yes	Yes	Yes (insertion order)
Typical AI use	batches of predictions	fixed shapes / constants	configs, feature dicts

✓ QUICK CHECK Strings & Data Structures

Q1. Why would you store an image's (height, width, channels) as a tuple instead of a list?

Answer: Because that shape shouldn't change by accident — a tuple's immutability protects against accidental edits.

Q2. What error do you get if you try to modify a tuple's element directly?

Answer: A `TypeError`, because tuples don't support item assignment.

Q3. How do you access the value 'NLP' from `model = {'name': 'GPT', 'task': 'NLP'}`?

Answer: `model['task']`

08

Conditions

```
x = 3
if (x == 2):
    print('Two')
elif (x == 3):
    print('Three')
else:
    print('others')
```

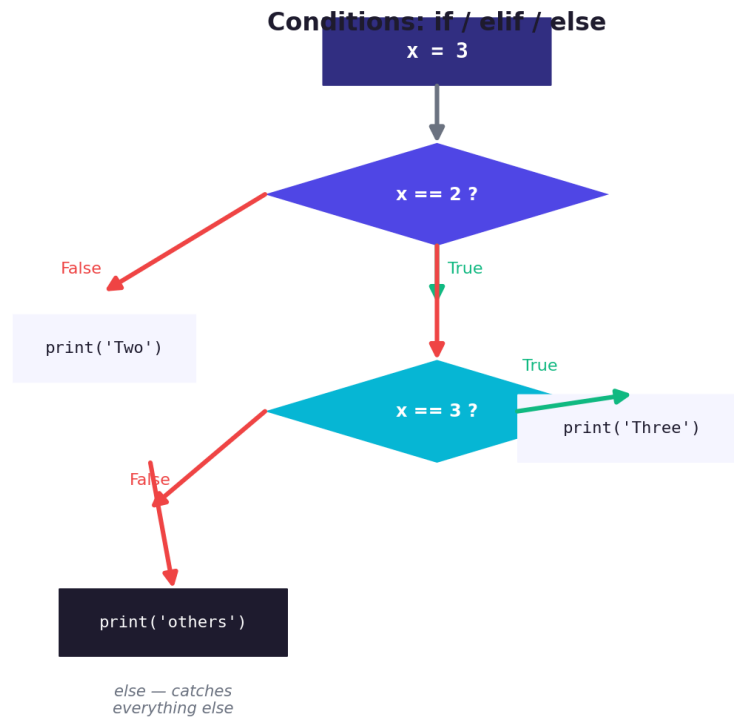


Figure 8.1 — Each condition is checked in order; the first True branch runs and the rest are skipped.

More condition examples

```
# A simple model-confidence gate
confidence = 0.62

if confidence >= 0.9:
    label = "high confidence"
elif confidence >= 0.6:
    label = "medium confidence"
else:
    label = "low confidence – flag for review"

print(label) # medium confidence

# Nested conditions: checking two things at once
age = 25
has_license = True
if age >= 18:
    if has_license:
        print("Can drive")
    else:
        print("Needs a license")
else:
    print("Too young")
```

Confidence thresholds like this are exactly how many ML pipelines decide when to auto-accept vs. flag a prediction.

✓ QUICK CHECK Conditions

Q1. In an if / elif / else chain, how many branches can run?

Answer: At most one — the first condition that evaluates to True runs, and the rest are skipped.

Q2. What does the else block do if none of the if/elif conditions match?

Answer: It runs as the fallback/default case.

Loops — for

```
languages = ['JavaScript', 'Python', 'Java']
for l in languages:
    print(l)
# JavaScript
# Python
# Java

# Looping over a dictionary of model metrics
metrics = {'accuracy': 0.91, 'precision': 0.88, 'recall': 0.85}
for name, value in metrics.items():
    print(f"{name}: {value}")
```

f-strings (f"...") are the modern, readable way to build strings with variables inside them.

Loops — while

```
dayCount = 0
while dayCount < 4:
    print('We are learning Python')
    dayCount += 1

# A "training loop" shape you'll see everywhere in ML code:
epoch = 0
loss = 1.0
while epoch < 5 and loss > 0.1:
    loss = loss * 0.6 # pretend the model is improving
    epoch += 1
    print(f"epoch {epoch}: loss={loss:.3f}")
```

Every training loop in PyTorch/TensorFlow is, underneath the framework code, a while or for loop like this.

Loops — break Statement

```
i = 1
while i < 6:
    if i == 3:
        break
    print(i)
    i += 1
# 1
# 2
```

Loops — continue Statement

```

i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
# 1
# 2
# 4
# 5
# 6

```

Loop Anatomy: while i < 6

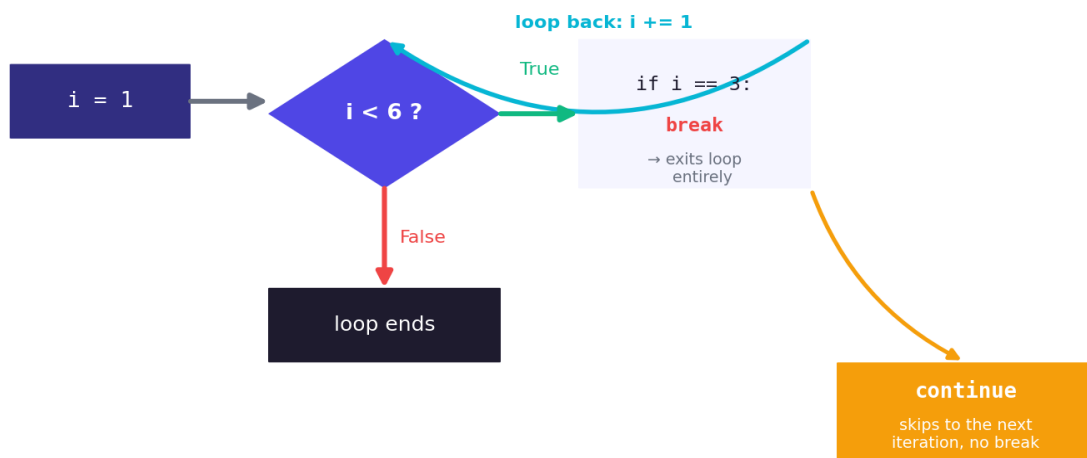


Figure 8.2 — `break` exits the loop entirely; `continue` only skips the current iteration.

✓ QUICK CHECK Loops

Q1. What is the key difference between `break` and `continue`?

Answer: `break` exits the loop completely; `continue` skips only the current iteration and moves to the next one.

Q2. Would a `for` loop or a `while` loop be more natural for 'repeat until the model's loss drops below 0.1'?

Answer: A `while` loop — the number of iterations isn't known in advance, only the stopping condition.

Q3. Trace this loop: `for i in range(3): print(i * 2)`. What prints?

Answer: 0, 2, 4 (on separate lines).

09

Python Levels

As programs grow — including AI pipelines — they evolve through these levels of organization:

Level	Description
Spaghetti Level	Unstructured code with jumps (go-to) between lines.
Procedural Level	Code organized using functions.
Modular Level	Code organized using modules.
Object Oriented Level	Code organized using classes and objects.

Python Levels — how programs grow up

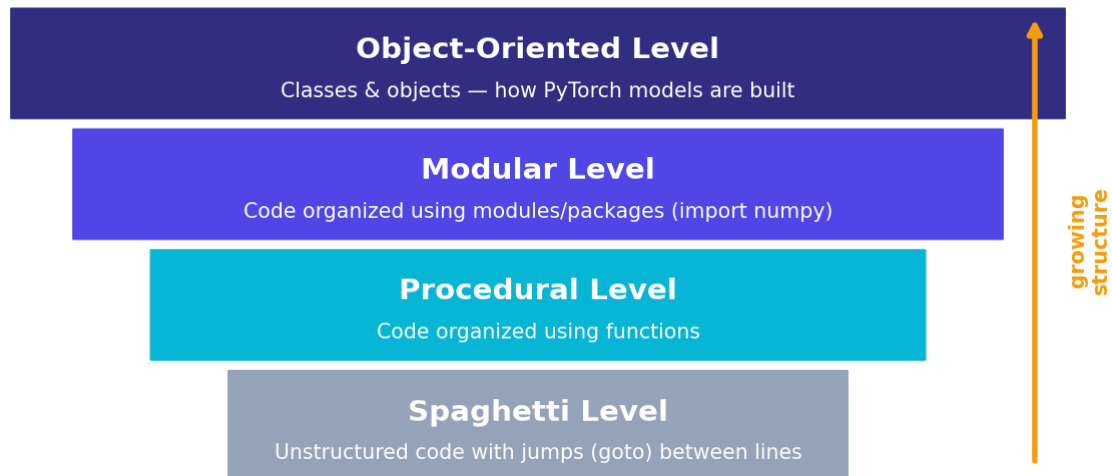


Figure 9.1 — Most real AI codebases live at the Modular or Object-Oriented level: think `import torch.nn as nn`, then `class MyModel(nn.Module)`.

AI CONNECTION A single-file experimental script (spaghetti/procedural) is fine for a quick test, but any shared research codebase — including a HAR classifier or a training pipeline — quickly needs modules and classes to stay maintainable.

✓ QUICK CHECK Python Levels

Q1. Which level uses functions to organize code but not yet classes?

Answer: *The Procedural Level.*

Q2. Why do most production ML frameworks (PyTorch, TensorFlow) rely on the Object-Oriented level?

Answer: *Because models, layers, and datasets are naturally represented as classes and objects with their own state and behavior.*

10

Function Declaration

A function is defined by its name, arguments, commands, and return value(s).

```
def measureTemp(temp):  
    if temp < 37:  
        return "Too Cold"  
    elif temp > 37:  
        return "Too Hot"  
    return "Normal"  
  
measureTemp(37)  
# "Normal"
```

Anatomy of a Function

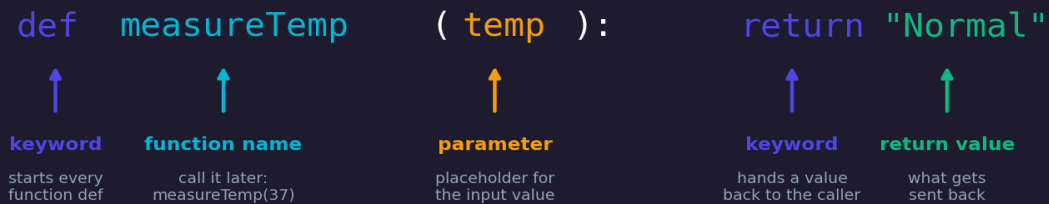


Figure 10.1 — Every function has the same four parts: keyword, name, parameter(s), and return value.

More function examples

```
# A function with no return value (returns None implicitly)
def log_message(msg):
    print(f"[LOG] {msg}")

log_message("Model loaded")    # [LOG] Model Loaded

# A function that returns multiple values (as a tuple)
def min_max(values):
    return min(values), max(values)

lo, hi = min_max([3, 7, 1, 9, 4])
print(lo, hi)    # 1 9

# A small, reusable preprocessing helper
def normalize(value, min_val, max_val):
    return (value - min_val) / (max_val - min_val)

print(normalize(75, 0, 100))    # 0.75
```

`normalize()` is a tiny version of the scaling step almost every ML pipeline applies to raw features.

Functions — Default Arguments

```
def doSum(x, y=2, z=3):
    total = x + y + z
    print(total)

doSum(2)          # output: 7
doSum(2, 4)       # output: 9
doSum(2, 4, 10)   # output: 16

# Default arguments are common for optional hyperparameters:
def train(model_name, epochs=10, learning_rate=0.001):
    print(f"Training {model_name} for {epochs} epochs at lr={learning_rate}")

train("QuantumStride")          # uses both defaults
train("QuantumStride", epochs=25)    # overrides just one
```

Default arguments let a function's caller override only what they need to, and leave sensible defaults alone.

Functions — *args

```
def doSum(*args):
    total = 0
    for i in args:
        total += i
    print(total)

doSum(2, 6)          # output: 8
doSum(2, 4, 5, 15)   # output: 26

# Handy for functions that average an unknown number of scores:
def average(*scores):
    return sum(scores) / len(scores)

print(average(0.8, 0.9, 0.75)) # 0.8166...
```

Functions — **kwargs

```
def doSum(**kwargs):
    for k in kwargs:
        print(kwargs[k])

doSum(x=2, y=26)
# 2
# 26

# Building a config dict from named arguments – very common in ML code:
def build_config(**kwargs):
    return kwargs

cfg = build_config(batch_size=32, learning_rate=0.001, epochs=10)
print(cfg) # {'batch_size': 32, 'learning_rate': 0.001, 'epochs': 10}
```

***kwargs is exactly how libraries like scikit-learn and PyTorch let you pass arbitrary named settings into a model.*

✓ QUICK CHECK Functions

Q1. What's the difference between *args and **kwargs?

Answer: *args collects extra positional arguments into a tuple; **kwargs collects extra named (keyword) arguments into a dictionary.

Q2. In def train(model_name, epochs=10): what happens if you call train('X')?

Answer: epochs takes its default value of 10, since no value was passed for it.

Q3. What does a function return if it has no explicit return statement?

Answer: None.

11

Commonly Used Functions

Input

```
# input(prompt_message)
name = input("What's your Name? ")
print(name)

# Reading a number safely (input() always returns a string!)
age_text = input("Enter your age: ")
age = int(age_text)
print(f"In 5 years you'll be {age + 5}")
```

A very common beginner bug: forgetting that `input()` always returns a string, even if the user types a number.

Range

```
# range([start,] end[, step])
range(5)           # [0, 1, 2, 3, 4]
range(0, 5, 1)     # [0, 1, 2, 3, 4]
range(1, 10, 2)    # [1, 3, 5, 7, 9]

for i in range(10):
    print(i)
# 0 1 2 3 4 5 6 7 8 9

# range() is how you'd loop over dataset indices without building a full list:
dataset_size = 5
for idx in range(dataset_size):
    print(f"Loading sample {idx}")
```

`range()` generates numbers lazily — it doesn't build the whole list in memory, which matters once datasets get large.

✓ QUICK CHECK Commonly Used Functions

Q1. What data type does `input()` always return, regardless of what the user types?

Answer: A string (`str`) — you must convert it (e.g. with `int()` or `float()`) to use it as a number.

Q2. What does `range(2, 11, 3)` produce?

Answer: 2, 5, 8 (starts at 2, stops before 11, steps by 3).

Next up: Lab 1

Put these fundamentals into practice — strings, arrays, sorting, conditionals, and pattern logic.